Compressão de Dados Científicos em GPUs

Conceitos, Ferramentas e Aplicações em HPC – Minicurso SSCAD 2026

B. O. T. Salvador[1] T. Maltempi[2] S. Rigo[2] H. Senger[3]

[1]Departamento de Computação, Universidade Federal de São Carlos (UFSCar)
[2]Instituto de Computação, Universidade Estadual de Campinas (Unicamp)
[3]Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo (USP)

2026

Link do Repositório do Minicurso



https://github.com/HPCSys-Lab/Minicurso_Compressao_SSCAD26

Roteiro

1 Motivação e Objetivos

2 Conceitos Básicos

3 Compressão em GPUs e Algoritmos

4 Métricas e Análise

5 APIs e Ferramentas

6 Conjunto de Dados e Atividade Prática

7 Conclusão

8 Referências

Roteiro

1 Motivação e Objetivos

2 Conceitos Básicos

3 Compressão em GPUs e Algoritmos

4 Métricas e Análise

5 APIs e Ferramentas

6 Conjunto de Dados e Atividade Prática

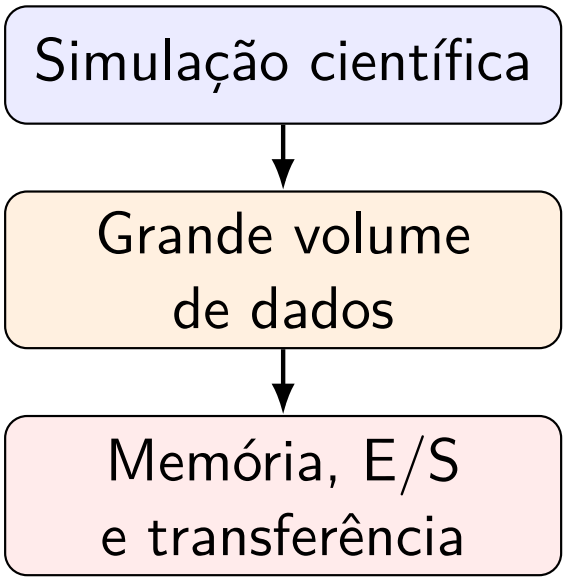
7 Conclusão

8 Referências

Motivação

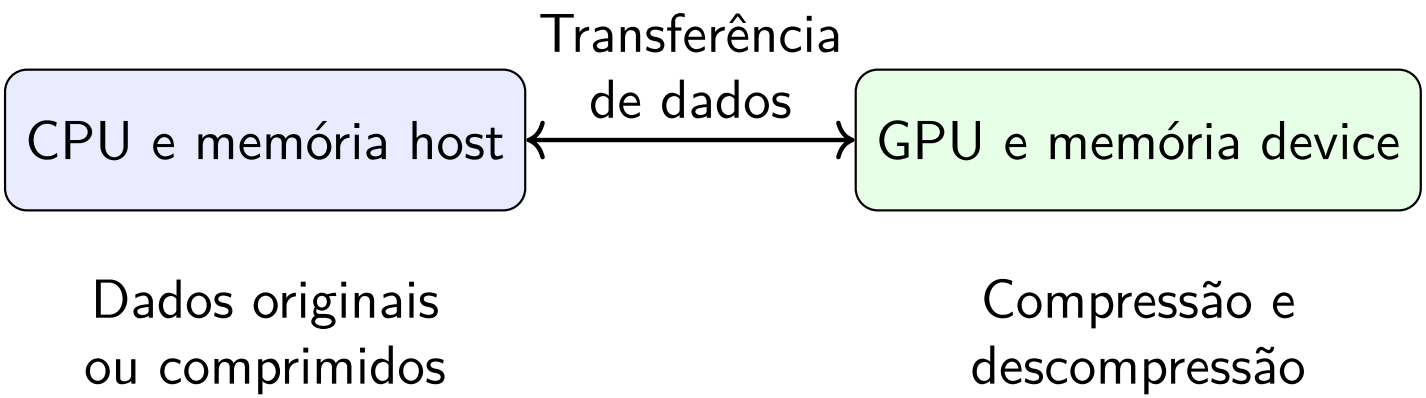
Aplicações científicas produzem grandes volumes de dados que precisam ser armazenados, transferidos ou reutilizados.

- Simulações climáticas e cosmológicas;
- Dinâmica de fluidos e combustão;
- Imageamento sísmico, RTM e FWI;
- Campos 3D e séries temporais longas.



Uso de GPU

- GPUs oferecem paralelismo e alta largura de banda de memória.
- O desempenho depende do algoritmo, dos acessos à memória, da sincronização e dos custos de transferência.
- A compressão pode reduzir a quantidade de dados movimentados, mas adiciona trabalho de compressão e descompressão.



Objetivos do minicurso

Ao final, espera-se que o participante possa:

1. Identificar aspectos de implementação de compressores em GPUs;
2. Reconhecer diferenças entre ZFP, cuSZp e nvCOMP Bitcomp;
3. Calcular métricas de tamanho, qualidade e desempenho;
4. Executar experimentos com dados científicos no Google Colab.

Roteiro

1 Motivação e Objetivos

2 Conceitos Básicos

3 Compressão em GPUs e Algoritmos

4 Métricas e Análise

5 APIs e Ferramentas

6 Conjunto de Dados e Atividade Prática

7 Conclusão

8 Referências

Entropia e redundância

- **Entropia:** mede a imprevisibilidade ou a aleatoriedade de uma fonte de dado.
- **Redundância:** padrões e correlações que permitem representar dados com menos bits.
- Entropia menor pode indicar maior possibilidade de codificação compacta sem perdas, conforme o modelo da fonte.

$$H(X) = -\sum_{i=1}^{n} p(x_i) \log_2 p(x_i)$$

A entropia é medida em bits por símbolo. (Shannon, 1948; Sayood, 2018)

Teorema da codificação da fonte

- Para uma fonte discreta sem memória, a entropia define um limite inferior para o número médio de bits por símbolo em codificação sem perdas.
- Um compressor sem perdas não pode reduzir indefinidamente o número médio de bits por símbolo abaixo desse limite, sob as hipóteses do modelo.
- E com isso exploramos a compressão com perdas e analisamos a viabilidade dela dentro de cada aplicação.

Compressão com e sem perdas

Sem perdas (*lossless*)

- Reconstrução bit a bit idêntica;
- Usada quando alterações não são permitidas;
- Em dados científicos de ponto flutuante, a redução pode ser limitada.

Com perdas (*lossy*)

- Permite maior redução quando a distorção é aceitável.
- Os dados reconstruídos podem diferir dos originais;
- A perda pode ser controlada por taxa ou tolerância de erro;

A aceitação da perda depende dos requisitos da aplicação científica.

Parâmetros de controle de perda de dados

Parâmetro	**O que controla**	Exemplo
Taxa fixa	Bits por valor	$rate = maxbits/4^d$ (ZFP)
Erro absoluto	Diferença absoluta permitida	Valor = 100, EB = 1e-4 = 0.0001
Erro relativo	Erro relacionado à escala dos dados	Valor = 100, EB = 1e-4 = 0.1

Roteiro

1 Motivação e Objetivos

2 Conceitos Básicos

3 Compressão em GPUs e Algoritmos

4 Métricas e Análise

5 APIs e Ferramentas

6 Conjunto de Dados e Atividade Prática

7 Conclusão

8 Referências

Arquitetura heterogênea e uso de GPU

CPU (host)

- Coordena a aplicação;
- Gerencia fluxo de controle e E/S;
- Pode preparar os dados para a GPU.

GPU (device)

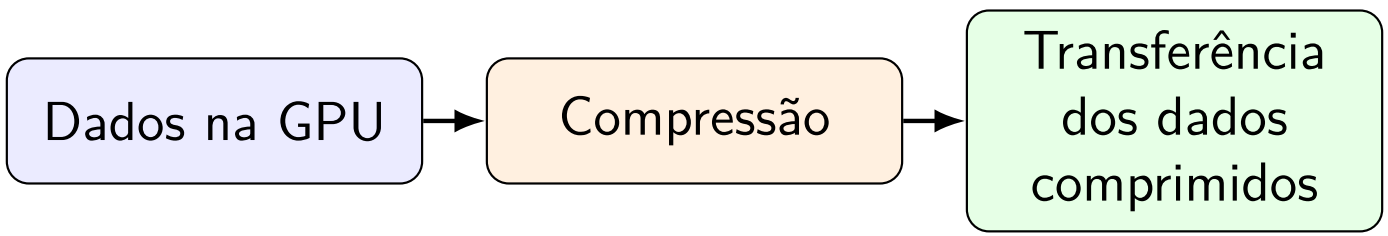
- Executa grande número de operações em paralelo;
- Tem alta largura de banda de memória;
- É adequada a operações paralelas por elemento ou bloco.

Aspectos que afetam o desempenho

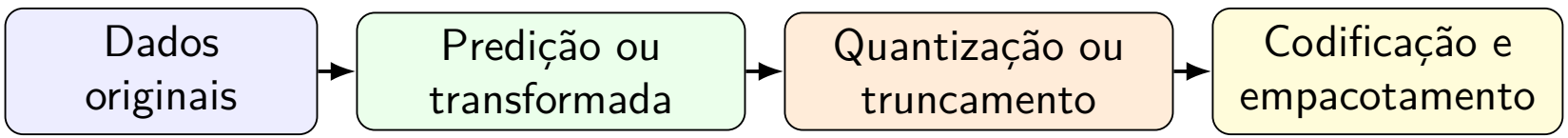
Divergência, acessos à memória, ocupação, transferências e custos de lançamento de kernels.

Localidade dos dados

- Dados precisam estar acessíveis à GPU antes da execução dos kernels.
- A compressão na GPU pode reduzir o volume transferido para CPU ou armazenamento.
- Para dados pequenos, transferências, alocações e sincronizações podem superar o custo de compressão.
- A avaliação deve registrar onde ocorrem compressão, descompressão e transferências.



Pipeline de compressores científicos



- Predição ou transformada explora correlações.
- A perda é introduzida pela quantização ou truncamento.
- Codificação e empacotamento geram os dados comprimidos.

Desafios de paralelização

1. **Saída de tamanho variável:** os blocos podem gerar diferentes quantidades de bytes.
2. **Dependências entre valores:** preditores podem depender de valores vizinhos.
3. **Acesso à memória:** acessos irregulares e divergência reduzem a eficiência.
4. **Lançamentos de kernels:** múltiplas etapas e buffers temporários aumentam o custo.

Uma abordagem comum é dividir os dados em blocos independentes e calcular posições globais para saídas de tamanho variável.

ZFP

- Comprime arranjos numéricos de uma a quatro dimensões em blocos de 4^d valores.
- Aplica transformação decorrelacionadora e codifica coeficientes em planos de bits.
- Modos: taxa fixa, precisão fixa, acurácia fixa e reversível.
- Na implementação CUDA do ZFP 1.0.1, o único modo suportado é o de taxa fixa, em que se seleciona $rate = maxbits/4^d$.

Execução CUDA

O tamanho de cada bloco é conhecido, mas a taxa fixa não garante um limite específico de erro.

cuSZp3

Modo	Estratégia	Uso inicial
`fixed`	Quantização e codificação por bloco, sem delta	Dados pouco correlacionados ou ruidosos
`plain`	Delta sensível à dimensão e codificação por bloco	Campos localmente suaves
`outlier`	Delta e tratamento separado de valores discrepantes	Campos suaves, não esparsos

- Processa campos 1D, 2D e 3D em FP32/FP64;
- A API utiliza erro absoluto; a CLI permite utilizar erro absoluto ou erro relativo.

Algoritmos NVIDIA nvCOMP

Propósito geral

- LZ4
- Snappy
- GDeflate
- Deflate
- Gzip
- Zstandard

Proprietários NVIDIA

- Cascaded: compressor de alto *throughput*, ideal para dados estruturados/tabulares ou analíticos
- ANS: codificador baseado em sistemas numéricos assimétricos (ANS)
- **Bitcomp**: compressor direcionado a aplicações de computação científica

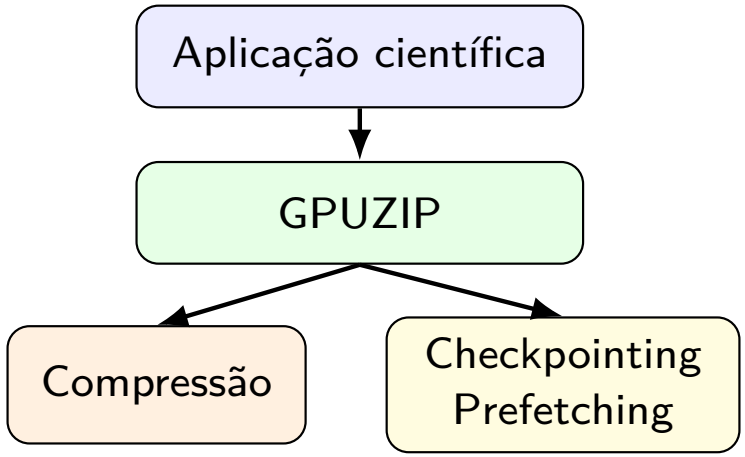
Comparação entre os compressores

Ferramenta	Modelo	Controle de perda
ZFP	DT+OWT+BPC	Taxa fixa (GPU)
cuSZp3	QT+PDP+LE	erro absoluto ou relativo
Bitcomp	Pipeline proprietário	Sem perdas / $\leq \delta/2$

- DT: transformada de domínio
- OWT: transformada wavelet ortogonal
- BPC: codificação bit-plane
- QT: quantização
- PDP: compressão baseada em predição
- LE: compressão sem perda (lossless)

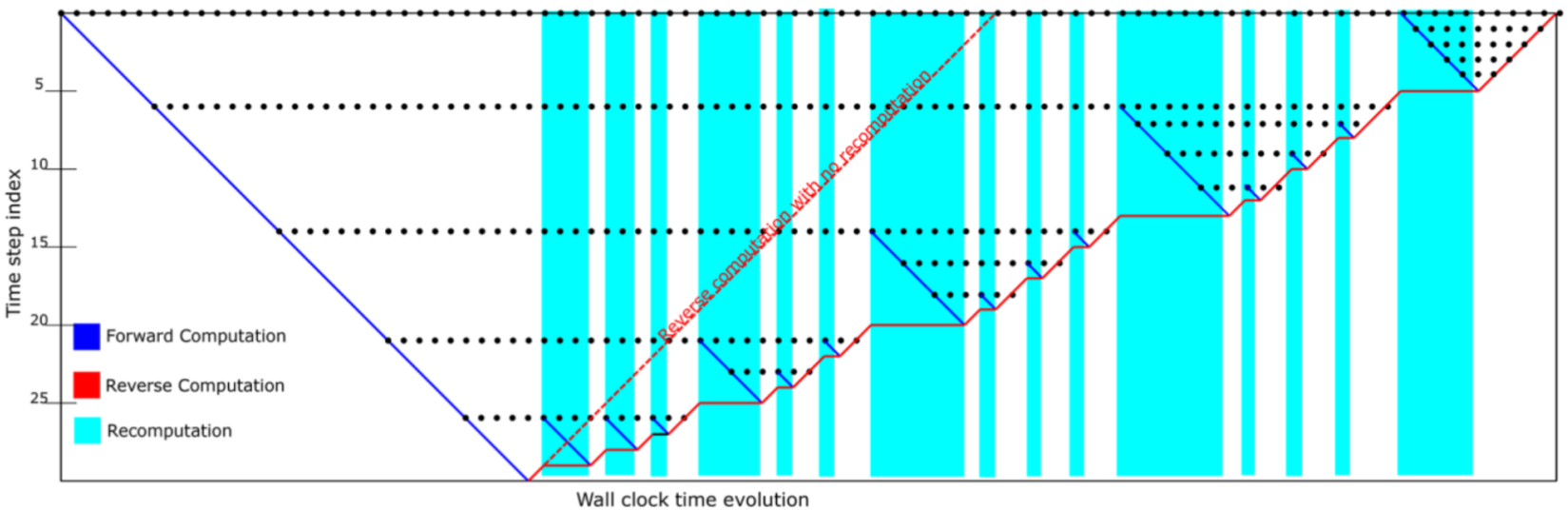
GPUZIP

- Biblioteca modular em C++/CUDA;
- Integra ZFP, cuSZp e nvCOMP Bitcomp;
- A camada de compressão pode ser usada de forma independente;
- Também oferece mecanismos de *checkpointing* e *prefetching*.

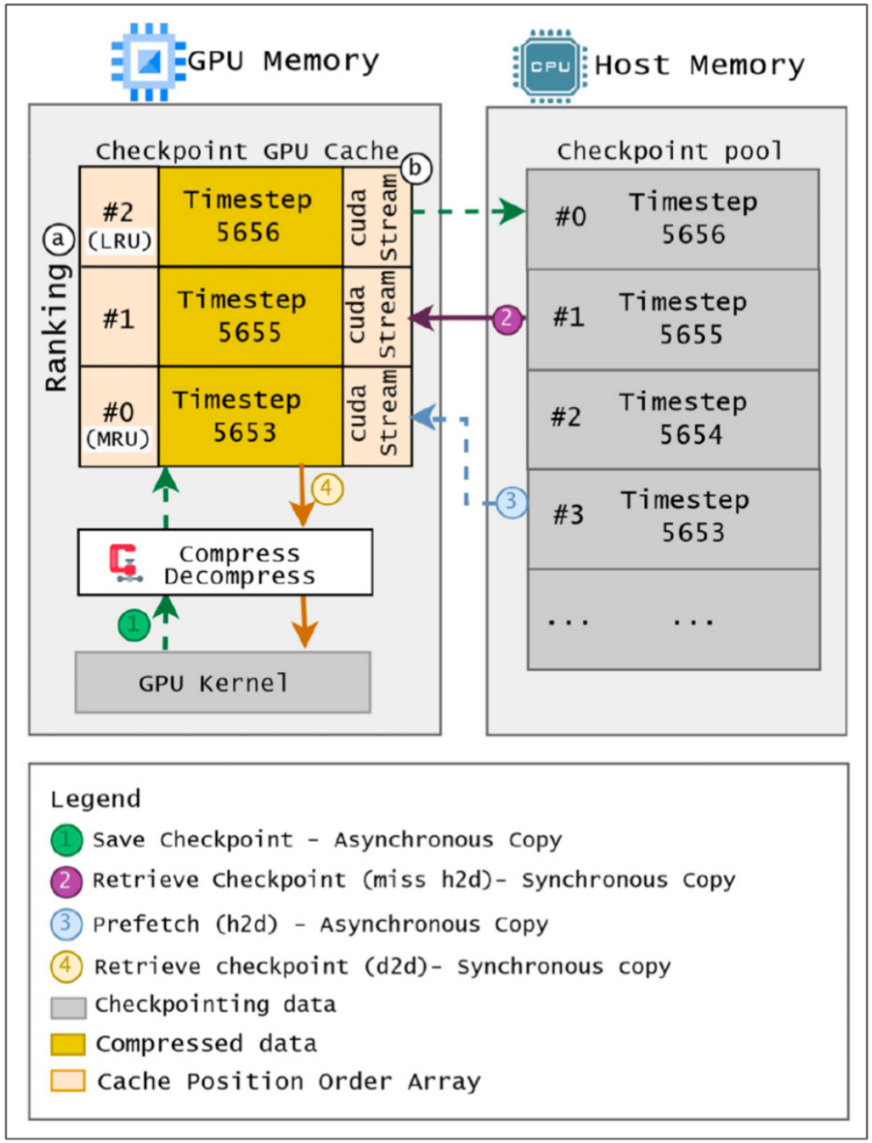


GPUZIPy disponibiliza a interface Python do módulo de compressão.

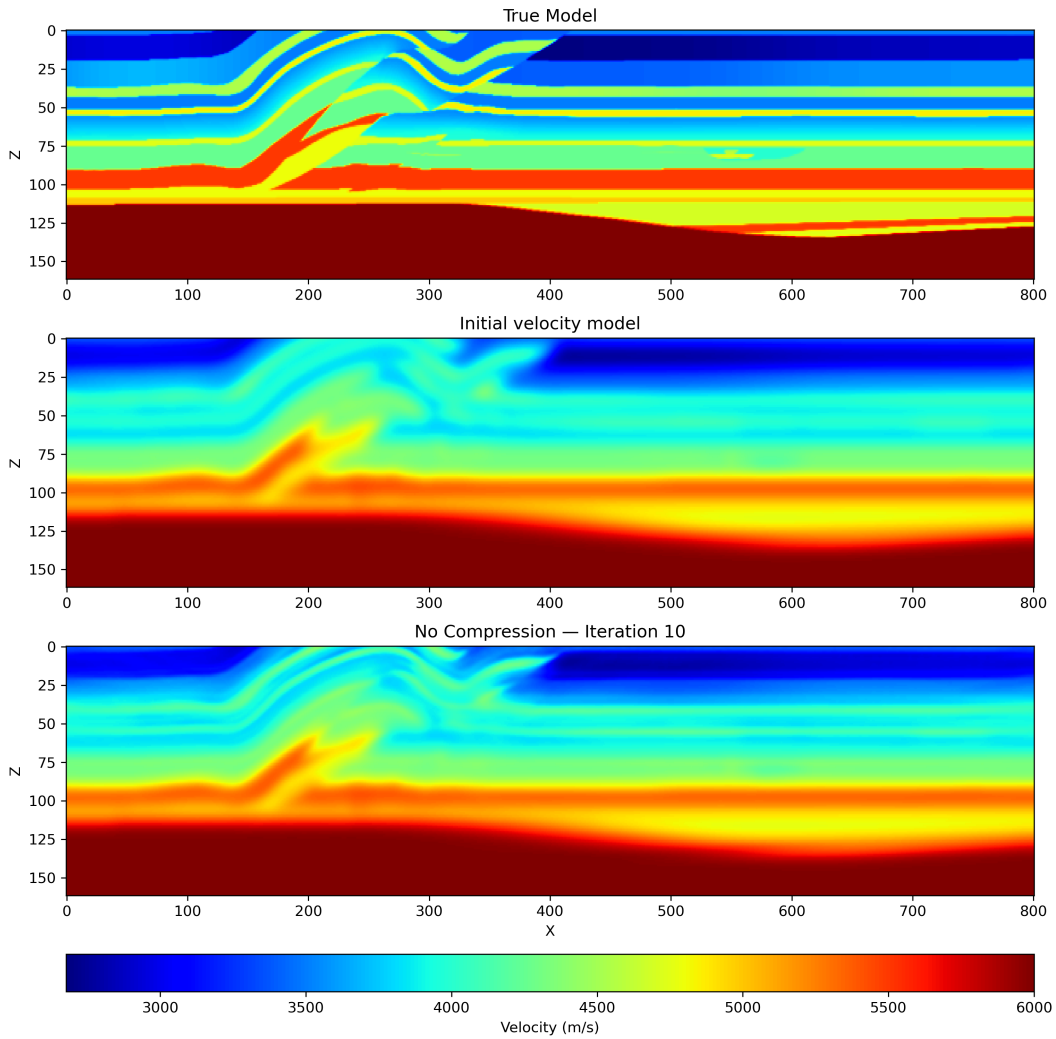
Checkpointing



GPUZIP 2.0 - Prefetcher



Exemplo: Full-Waveform Inversion (FWI)



Roteiro

1 Motivação e Objetivos

2 Conceitos Básicos

3 Compressão em GPUs e Algoritmos

4 Métricas e Análise

5 APIs e Ferramentas

6 Conjunto de Dados e Atividade Prática

7 Conclusão

8 Referências

Mtricas de compressão: taxa de compressão e bit rate

Taxa de compressão (CR)

$$CR = \frac{S_{\text{original}}}{S_{\text{comprimido}}}$$

$CR = 4$ significa que o original é quatro vezes maior que a representação comprimida.

Bit rate

$$b = \frac{S_{\text{comprimido}}}{N} \quad \text{bytes por valor}$$

Para dados FP32, 1 byte por valor corresponde nominalmente a um $CR \approx 4$ (32 bits divididos por 8 bits), sem considerar custos adicionais.

Métricas de qualidade: erro absoluto máximo, MSE e RMSE

Considere os dados originais x_i e reconstruídos $\hat{x}_i$.

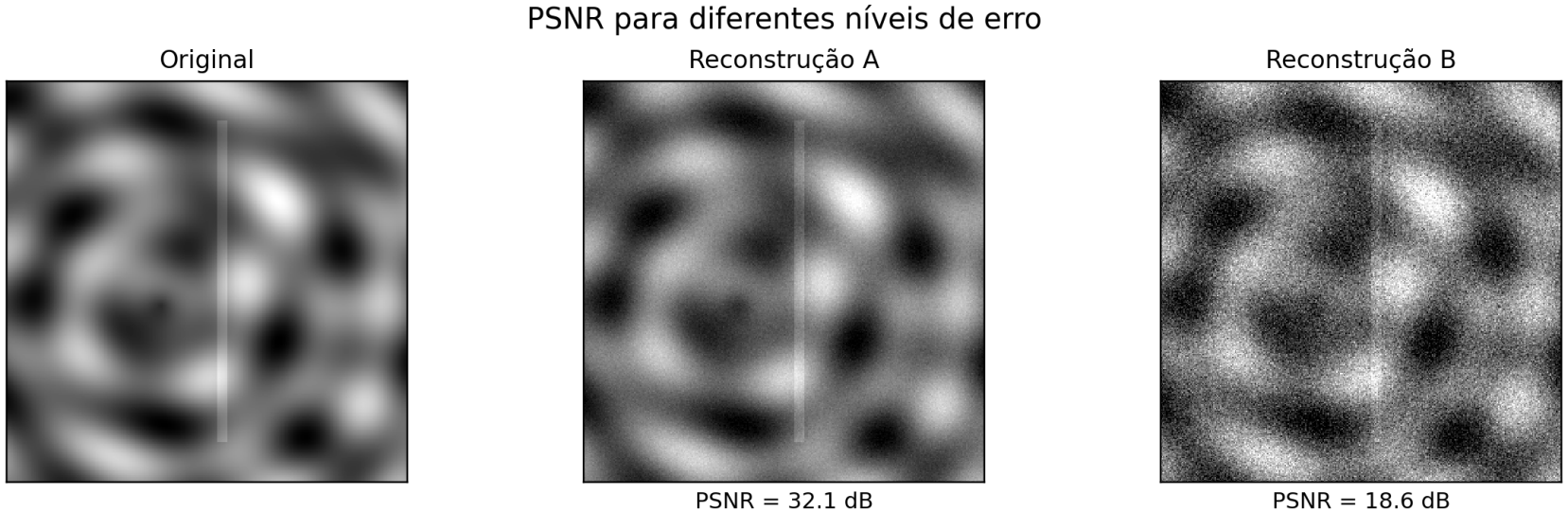
$$E_{\max} = \max_i |x_i - \hat{x}_i|$$

$$\mathrm{MSE} = \frac{1}{N} \sum_{i=1}^{N} (x_i - \hat{x}_i)^2$$

$$\mathrm{RMSE} = \sqrt{\mathrm{MSE}}$$

- $E_{\max}$ mede a maior diferença observada;
- MSE penaliza mais os erros de maior magnitude;
- RMSE está na escala dos dados originais.

Métricas de qualidade: PSNR



$$\mathrm{PSNR} = 10 \log_{10} \left(\frac{M^2}{\mathrm{MSE}} \right) \ \mathrm{dB}$$

M é a amplitude de referência. A figura usa um campo sintético normalizado e mostra que menor MSE resulta em PSNR maior.

Métricas de qualidade: SSIM



O SSIM compara luminância, contraste e estrutura local. A figura mostra exemplos sintéticos; o valor depende dos dados e da configuração de cálculo.

Métricas de desempenho

$$T_{\text{total}} = T_{\text{compressão}} + T_{\text{descompressão}}$$

$$V_c = \frac{S_{\text{entrada}}}{T_c}, \qquad V_d = \frac{S_{\text{reconstruído}}}{T_d}$$

- T_{total} é o tempo total usado para todo o fluxo de compressão e descompressão, considerando alocação de memória, cópias, E/S e sincronização
- V_c é a vazão ou *throughput* de compressão
- V_d é a vazão ou *throughput* de descompressão

Roteiro

1 Motivação e Objetivos

2 Conceitos Básicos

3 Compressão em GPUs e Algoritmos

4 Métricas e Análise

5 APIs e Ferramentas

6 Conjunto de Dados e Atividade Prática

7 Conclusão

8 Referências

Interfaces disponíveis

Ferramenta	CLI	C/C++	Python
cuSZp3	Sim	Sim	Sim
ZFP	Sim	Sim	Sim (CPU)
nvCOMP Bitcomp	Não	Sim	Sim
GPUZIP / GPUZIPy	Não	Sim	Sim

Tabela: Interfaces disponíveis para as ferramentas de compressão em GPU.

CLI: cuSZp3

```
./install/bin/cuSZp \
  -i campo.f32 -t f32 -m plain \
  -d 2 1800 3600 \
  -eb abs 1E-4 \
  -x campo.cuszp.cmp -o campo.cuszp.dec
```

- `-i`: entrada; `-t`: tipo; `-m`: modo;
- `-d`: dimensionalidade e dimensões;
- `-eb abs`: limite de erro absoluto;
- `-x`: saída comprimida; `-o`: reconstrução.

O arquivo de entrada deve corresponder ao tipo e às dimensões indicados.

CLI: ZFP com CUDA

```
./build/bin/zfp \
  -x cuda \
  -i campo.f32 -z campo.zfp \
  -f -2 1800 3600 -r 8 -s
```

- `-x cuda`: seleciona execução CUDA;
- `-f -2`: campo 2D de valores float;
- `-r 8`: taxa fixa de 8 bits por valor;
- `-s`: solicita estatísticas de erro.

No ZFP 1.0.1, a execução CUDA suporta taxa fixa, que não estabelece um limite absoluto de erro. (LLNL, 2024)

API C/C++: cuSZp3

```
// Os buffers d_original, d_compressed e d_reconstructed
// devem estar alocados na GPU e inicializados conforme necessario.
uint3 dims = make_uint3(nx, ny, nz);
size_t compressed_size = 0;

cuSZp_compress_3D_plain_f32(
    d_original, d_compressed, num_elements,
    &compressed_size, dims, error_bound, stream);

cuSZp_decompress_3D_plain_f32(
    d_reconstructed, d_compressed, num_elements,
    compressed_size, dims, error_bound, stream);
cudaStreamSynchronize(stream);
```

A capacidade do buffer comprimido deve ser dimensionada de acordo com a API e a versão utilizadas. (Scientific Data Compression Group, 2025)

API C/C++: ZFP com CUDA

```
zfp_field *field_in = zfp_field_3d(
    original, zfp_type_float, nx, ny, nz);
zfp_field *field_out = zfp_field_3d(
    reconstructed, zfp_type_float, nx, ny, nz);
zfp_stream *zfp = zfp_stream_open(NULL);

zfp_stream_set_rate(zfp, 8, zfp_type_float, 3, zfp_false);
zfp_stream_set_execution(zfp, zfp_exec_cuda);
/* Configurar buffer e bitstream antes das chamadas. */
zfp_compress(zfp, field_in);
zfp_stream_rewind(zfp);
zfp_decompress(zfp, field_out);
```

O exemplo pressupõe a configuração prévia do buffer e do fluxo de bits. Para CUDA, use taxa fixa na versão 1.0.1. (LLNL, 2024)

API C/C++: nvCOMP Bitcomp

```
bitcompHandle_t plan;
bitcompCreatePlan(&plan, input_bytes,
    BITCOMP_FP32_DATA,
    BITCOMP_LOSSY_FP_TO_SIGNED,
    BITCOMP_DEFAULT_ALGO);
bitcompSetStream(plan, stream);

bitcompCompressLossy_fp32(
    plan, d_original, d_compressed, delta);
cudaStreamSynchronize(stream);
bitcompUncompress(plan, d_compressed, d_reconstructed);
cudaStreamSynchronize(stream);
bitcompDestroyPlan(plan);
```

A API nativa do Bitcomp pode variar entre versões; consulte a documentação da versão instalada. (NVIDIA, 2026)

API Python: cuSZp3

```
from pycuSZp import cuSZp
import ctypes

compressor = cuSZp()
dims = (nz, ny, nx) # ordem (z, y, x)
compressed_size = compressor.compress(
    ctypes.c_void_p(int(d_original)),
    ctypes.c_void_p(int(d_compressed)),
    data.size, error_bound,
    dim=3, dims=dims, data_type=0, mode=1)
compressor.decompress(
    ctypes.c_void_p(int(d_reconstructed)),
    ctypes.c_void_p(int(d_compressed)),
    data.size, compressed_size, error_bound,
    dim=3, dims=dims, data_type=0, mode=1)
```

Os ponteiros referem-se a buffers CUDA previamente alocados; a capacidade do buffer comprimido deve ser suficiente. (Scientific Data Compression Group, 2025)

API Python: nvCOMP Bitcomp

```
import cupy as cp
from nvidia import nvcomp

# original: array NumPy de valores float32
nv_original = nvcomp.as_array(cp.asarray(original))
codec = nvcomp.Codec(
    algorithm="Bitcomp", data_type="<f4",
    mode=nvcomp.BitcompMode.BITCOMP_LOSSY_FP_TO_SIGNED,
    delta=2.0e-4)
compressed = codec.encode(nv_original)
reconstructed_gpu = codec.decode(compressed, data_type="<f4")
reconstructed = reconstructed_gpu.cpu()
```

Este exemplo usa a API Python atual; os parâmetros e a instalação devem corresponder à versão do nvCOMP. (NVIDIA, 2026)

Roteiro

1 Motivação e Objetivos

2 Conceitos Básicos

3 Compressão em GPUs e Algoritmos

4 Métricas e Análise

5 APIs e Ferramentas

6 Conjunto de Dados e Atividade Prática

7 Conclusão

8 Referências

SDRBench

O *Scientific Data Reduction Benchmarks* reúne conjuntos de dados científicos de diferentes áreas para avaliação de técnicas de redução de dados. (SDRBench, 2026)

Dataset	**Área**	**Dimensões**	**Precisão**
CESM-ATM	Clima	1800×3600	FP32
NYX	Cosmologia	$512 \times 512 \times 512$	FP32
Hurricane Isabel	Meteorologia	$100 \times 500 \times 500$	FP32

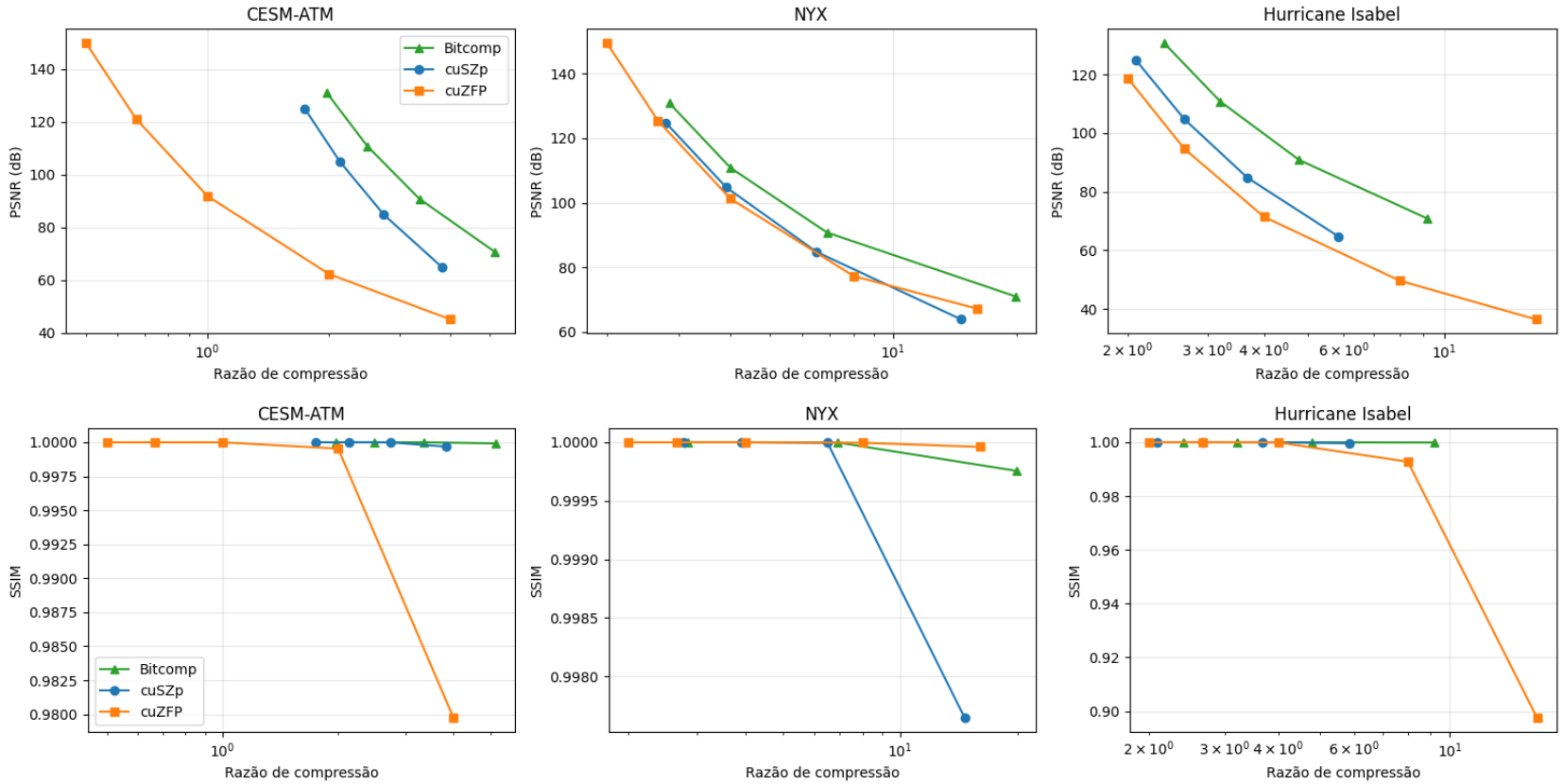
Catálogo: SDRBench

Comparação Entre Compressores: Parâmetros e Teste

- cuSZp e nvCOMP Bitcomp: erros de 1e-3, 1e-4, 1e-5 e 1e-6
- ZFP: bit rate de 2, 4, 8, 12 e 16

Comparação Entre Compressores: Qualidade

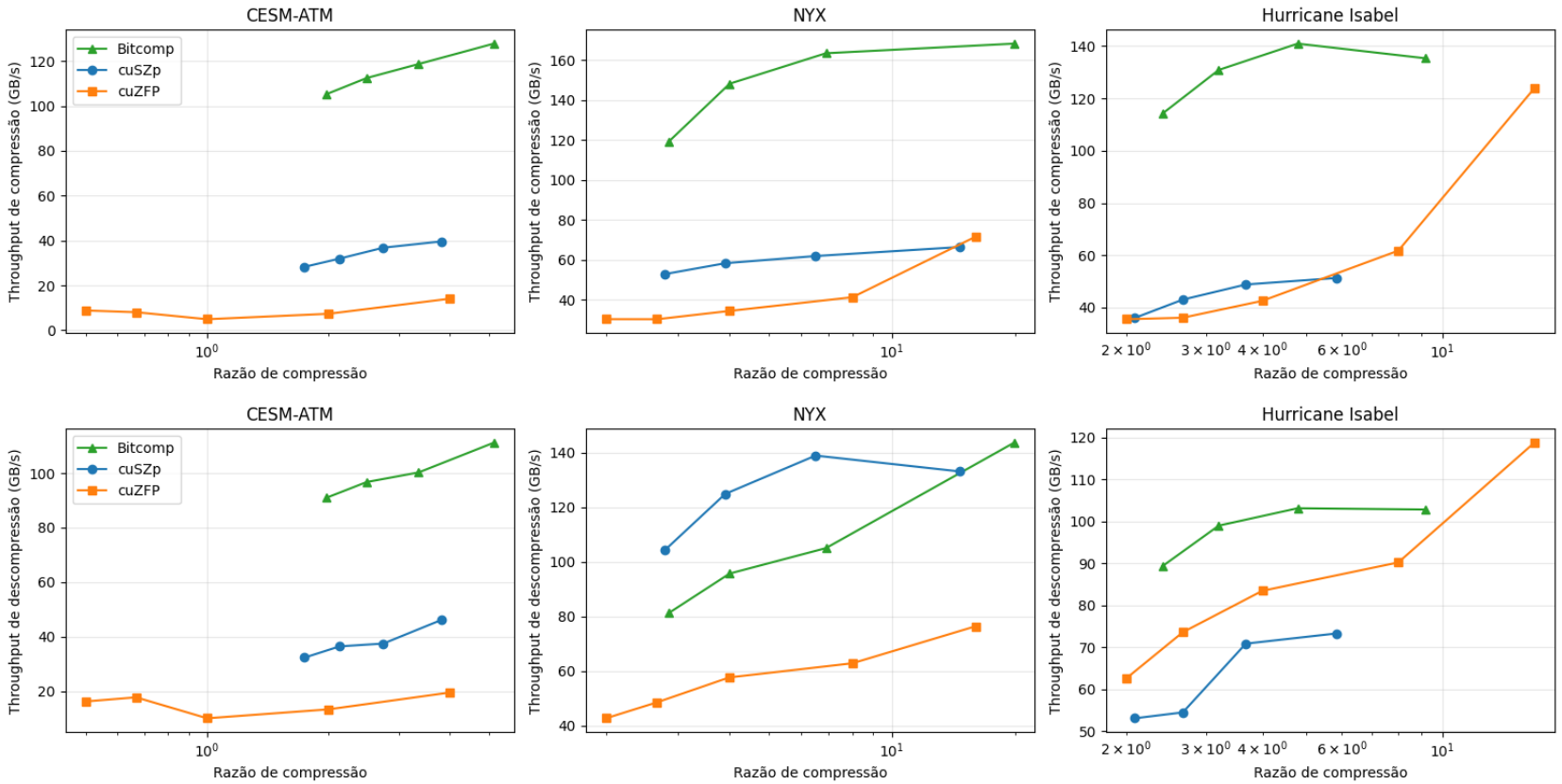
PSNR (em cima) e SSIM (embaixo) em função da razão de compressão para cuSZp, ZFP e nvCOMP Bitcomp nos datasets CESM-ATM, NYX e Hurricane Isabel.



GPU NVIDIA T4 (Google Colab); dados em FP32.

Comparação Entre Compressores: Vazão (throughput)

Throughput de compressão (em cima) e de descompressão (embaixo), em GB/s, em função da razão de compressão, medido em uma GPU T4 no Google Colab.



Atividade 1: cuSZp3 e ZFP no Google Colab

Verificar se a instância do Google Colab está conectada a uma GPU NVIDIA T4. É possível replicar o experimento caso sua máquina tenha uma GPU NVIDIA dedicada.

Notebook

Abrir notebook do cuSZp3

Atividade 2: GPUZIPy

Notebook

Verificar se atende aos requisitos semelhantes aos do slide anterior.
Abrir notebook de exemplo do GPUZIPy

A disponibilidade dos backends depende da GPU, CUDA e versões instaladas.

Interpretação dos resultados

1. Qual foi a taxa de compressão?
2. Qual foi o erro máximo e o RMSE após a reconstrução?
3. A compressão foi executada na GPU?
4. O tempo inclui transferências, alocações e sincronização? (questionamento)
5. A economia de armazenamento compensa o custo adicional? (questionamento)

Roteiro

1 Motivação e Objetivos

2 Conceitos Básicos

3 Compressão em GPUs e Algoritmos

4 Métricas e Análise

5 APIs e Ferramentas

6 Conjunto de Dados e Atividade Prática

7 Conclusão

8 Referências

Conclusão

- Compressão pode reduzir memória, armazenamento e movimentação de dados.
- O uso de GPU não elimina custos de transferência, sincronização e buffers.
- ZFP, cuSZp3 e Bitcomp utilizam modelos de compressão e parâmetros de controle diferentes.
- A avaliação da viabilidade do compressor deve incluir taxa de compressão, qualidade da reconstrução e desempenho.
- A adequação depende das características dos dados e dos requisitos da aplicação.

Roteiro

1 Motivação e Objetivos

2 Conceitos Básicos

3 Compressão em GPUs e Algoritmos

4 Métricas e Análise

5 APIs e Ferramentas

6 Conjunto de Dados e Atividade Prática

7 Conclusão

8 Referências

Referências (1/2)

SHANNON, C. E. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423, 1948.

SAYOOD, K. *Introduction to Data Compression*. 5th edition. Morgan Kaufmann, 2018.

LINDSTROM, P. Fixed-rate compressed floating-point arrays. *IEEE Transactions on Visualization and Computer Graphics*, 20(12):2674–2683, 2014.

DI, S. et al. A survey on error-bounded lossy compression for scientific datasets. *ACM Computing Surveys*, 57(11), 2025.

HUANG, Y. et al. cuSZp: An ultra-fast GPU error-bounded lossy compression framework with optimized end-to-end performance. In *Proceedings of SC*, pages 1–13, 2023.

HUANG, Y. et al. cuSZp2: A GPU lossy compressor with extreme throughput and optimized compression ratio. In *Proceedings of SC*, pages 1–18, 2024.

Referências (2/2)

HUANG, Y. et al. GPU lossy compression for HPC can be versatile and ultra-fast. In *Proceedings of SC*, pages 2056–2075, 2025.

NVIDIA. *nvCOMP: GPU Compression Library*. Documentation. https://docs.nvidia.com/cuda/nvcomp/index.html.

Lawrence Livermore National Laboratory. *ZFP Documentation, Release 1.0.1*. https://zfp.readthedocs.io/en/release1.0.1/.

Scientific Data Compression Group. *cuSZp: GPU Error-Bounded Lossy Compressor*. https://github.com/szcompressor/cuSZp.

MALTEMPI, T. et al. Checkpointing fine-tuning for accelerating seismic applications in GPUs. *International Journal of High Performance Computing Applications*, 39(6):784–802, 2025.

LSC-Unicamp. *GPUZIP: GPU-based data compression and checkpointing*. https://github.com/LSC-Unicamp/GPUZIP.

SDRBench. *Scientific Data Reduction Benchmarks: Datasets*. https://sdrbench.github.io/datasets.html.

Obrigado!

Obrigado pela participação!